

Tema 5 – Confección d Informes

Contenido

1. Introducción a la generación de informes en aplicaciones multiplataforma	1
2. WPF – Generación de informes con RDLC Reports	2
3.1 Estructura de un informe RDLC	2
3.2 Creación de un informe RDLC	2
3.3 Elementos avanzados	2
4. Android – Generación de informes PDF con iText7	3
4.1 iText7 en Android	3
4.2 Paso a paso (proyecto “Jugando con botones” o “Formulario de Empleados”).....	3
4.3 Elementos avanzados	4

1. Introducción a la generación de informes en aplicaciones multiplataforma

Un **informe** es un documento estructurado y visual que presenta información procesada de una fuente de datos (base de datos, archivos, etc.) de forma clara, legible y profesional.

El objetivo es integrar la generación de informes directamente dentro de la interfaz de la aplicación que estamos desarrollando (WPF o Android). De esta forma, el usuario final puede generar y visualizar el informe con un solo clic desde la propia aplicación.

¿Por qué es importante en el mundo profesional?

- Los clientes y empresas necesitan informes claros y actualizados en tiempo real.
- Un informe bien diseñado mejora la experiencia de usuario (UX) y la usabilidad de la aplicación.
- Cumple con los requisitos de muchos proyectos reales: facturas, nóminas, listados de stock, resúmenes de ventas, etc.

Las herramientas recomendadas son:

Plataforma	Herramienta principal	Características
WPF	RDLC Reports + ReportViewer	Integración nativa con Visual Studio, asistente gráfico, conexión directa a SQL
Android	iText7 para Kotlin o PdfDocument nativo	Generación de PDF profesional, ligera, se empaqueta en el APK

2. WPF – Generación de informes con RDLC Reports

3.1 Estructura de un informe RDLC

Un informe RDLC se organiza en secciones claras:

- **Encabezado (Header):** Cabecera repetitiva (logo, título, fecha).
- **Cuerpo (Body):** Tablas, gráficos, subtotales y totales.
- **Pie de página (Footer):** Información de página, copyright, etc.
- **Parámetros y filtros:** Permiten personalizar el informe (fecha, departamento, cliente...).

3.2 Creación de un informe RDLC

1. En tu proyecto WPF y añade el control ReportViewer al formulario (desde la caja de herramientas).
2. Crea un nuevo informe: **Proyecto → Agregar → Nuevo elemento → Report (RDLC)**.
3. Configura el origen de datos (Data Source) conectándote a tu base de datos (ya vista en UD4).
4. Usa el asistente gráfico para diseñar:
 - Arrastra campos a la tabla.
 - Añade agrupamientos y subtotales.
 - Inserta gráficos o imágenes.
5. Define parámetros (ej.: fecha de inicio y fin).
6. Muestra el informe en la aplicación con código similar a este:

```
private void btnGenerarInforme_Click(object sender, RoutedEventArgs e)
{
    reportViewer1.LocalReport.ReportPath = "NombreDelInforme.rdlc";
    reportViewer1.LocalReport.DataSources.Clear();
    reportViewer1.LocalReport.DataSources.Add(
        new ReportDataSource("NombreDataSet", listaDeDatos));
    reportViewer1.RefreshReport();
}
```

3.3 Elementos avanzados

- Agrupamientos y subtotales con expresiones: =Sum(Fields!Importe.Value).
- Gráficos (Chart) y expresiones calculadas.
- Imágenes dinámicas (logo de empresa).
- Exportación automática a PDF, Excel o Word desde código.
- Usa el modo Design para ver el resultado en tiempo real y evita errores de diseño.

4. Android – Generación de informes PDF con iText7

4.1 iText7 en Android

- Es ligero y gratuito.
- Se incluye completamente en el APK (no depende de servicios externos).
- Permite total control: tablas, imágenes, gráficos, encabezados y firmas.
- Cumple perfectamente con el enfoque competencial del módulo (generar informe dentro de la app).

4.2 Creación de un informe PDF

Añadir dependencia en build.gradle.kts (app) en la parte de Kotlin:

```
implementation("com.itextpdf:itext7-core:8.0.5")
```

Código ejemplo para generar informe PDF:

- Creación del PDF (líneas 3-5)
 - PdfWriter: es el “escritor” que va a guardar el archivo en disco.
 - PdfDocument: es el documento interno de iText7.
 - Document: es la capa más cómoda (la que usamos para añadir texto, tablas, imágenes...).
- Añadir contenido (líneas 8-20)
 - document.add(...): es el método principal para insertar cualquier elemento (párrafos, tablas, imágenes...).
 - La tabla se crea con Table(númeroDeColumnas) y luego se van añadiendo celdas.
- Cerrar el documento (línea 22)
 - **Obligatorio.** Si no cierras el documento, el PDF puede quedar corrupto o vacío.
 - Abrir/compartir el PDF (líneas 25-33)
- FileProvider: necesario por seguridad en Android 7 o superior.
- Intent.ACTION_VIEW: abre el PDF con la aplicación predeterminada del dispositivo (Adobe, Google Drive, etc.).

```
private fun generarInformePDF() {  
    val pdfWriter = PdfWriter("Informe.pdf")  
    val pdfDocument = PdfDocument(pdfWriter)  
    val document = Document(pdfDocument)  
  
    // Cabecera  
    document.add(Paragraph("Informe General").setFontSize(20f))  
  
    // Tabla de datos  
    val table = Table(4) // 4 columnas  
    // Añadir filas con datos...  
    document.add(table)  
  
    // Totales y pie  
    document.add(Paragraph("Total: XXXX").setFontSize(14f))  
  
    document.close()  
  
    // Abrir o compartir el PDF  
    val uri = FileProvider.getUriForFile(this, "${packageName}.provider",  
    File("Informe.pdf"))  
    val intent = Intent(Intent.ACTION_VIEW).apply {  
        setDataAndType(uri, "application/pdf")  
        addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)  
    }  
    startActivity(intent)  
}
```

4.3 Elementos avanzados

- Parámetros y filtros (usando consultas Room con @Query).
- Gráficos (MPAndroidChart o Compose Charts exportados a PDF).
- Inclusión de imágenes y logos.
- Firma digital del PDF (ampliación).
- Ejecutar siempre en un hilo secundario (coroutine o Thread) para evitar bloquear la interfaz.